

P1144R7

std::is_trivially_relocatable

Published Proposal, 2023-03-10

Issue Tracking:

[Inline In Spec](#)

Author:

[Arthur O'Dwyer](#)

Audience:

LEWG, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:

42

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1144-object-relocation.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1144-object-relocation.html

Abstract

We define a new verb, "relocate," equivalent to a move and a destroy. For many C++ types, the "relocate" operation is implementable as a single `memcpy`. We provide a standard trait to detect types which are trivially relocatable, for the benefit of library writers. We provide a rule that correctly propagates trivial relocatability among Rule-of-Zero types. Finally, we provide a portable way for a user-defined type (e.g. `boost::shared_ptr`) to warrant that it is trivially relocatable. P1144R3 was presented at C++Now 2019 as a 90-minute talk titled "[Trivially Relocatable](#)."

Table of Contents

- 1 Changelog**
- 2 Introduction and motivation**
 - 2.1 Optimizations enabled by trivial relocatability
 - 2.1.1 Contiguous reallocation
 - 2.1.2 Moving in-place/SBO type-erased types like `any` and `function`
 - 2.1.3 Moving fixed-capacity containers like `static_vector` and `small_vector`
 - 2.1.4 Contiguous insert and erase
 - 2.1.5 Swap
 - 2.2 Assertions, not assumptions
 - 2.3 Eliminate manual warrants, improve safety
- 3 Design goals**
 - 3.1 Standard library types such as `std::string`
 - 3.2 Program-defined types that follow the Rule of Zero
 - 3.3 Program-defined types with non-defaulted special members
- 4 Proposed wording for C++2b**
 - 4.1 Nothrow bidirectional iterator [algorithms.requirements]
 - 4.2 Relocation operation [defs.relocation]

- 4.3 [relocate_at](#) and [relocate](#) [specialized.relocate]
- 4.4 [uninitialized_relocate](#), [uninitialized_relocate_n](#), [uninitialized_relocate_backward](#) [uninitialized.relocate]
- 4.5 Trivially relocatable type [basic.types.general]
- 4.6 `[[trivially\_relocatable]]` attribute [dcl.attr.trivrelloc]
- 4.7 Type traits [is_relocatable](#) etc. [meta.unary.prop]
- 4.8 [relocatable](#) concept [concept.relocatable]

5 Rationale and alternatives

- 5.1 Attribute `[[maybe\_trivially\_relocatable]]`
- 5.2 Confusing interactions with `std::pmr`
- 5.3 Overlapping base-class subobjects

6 Acknowledgements

Appendix A: Straw polls

Polls taken at EWGI at Issaquah on 2023-02-10

Polls taken at EWGI at Prague on 2020-02-13

Appendix B: Sample code

Appendix C: Examples of non-trivially relocatable class types

Class contains pointer to self

Allocated memory contains pointer to self

Class invariant depends on `this`

Program invariant depends on `this`

Polymorphic downcast effectively relies on offset-into-self

Appendix D: Implementation experience

Appendix E: Open questions

Index

Terms defined by this specification

References

Normative References

Informative References

Issues Index

§ 1. Changelog

- R7 (post-Issaquah 2023):
 - [\[P2786R0\]](#) debuted at Issaquah; it raised several open questions (see [Appendix E: Open questions](#)). The authors of P1144 and P2786 plan a joint paper to resolve some of these design issues.
 - Added `std::uninitialized_relocate_backward`, the building block of `vector::insert`.
 - Removed "move-constructible, destructible" from the requirements in [basic.types.general](#). Now trivially relocatable types, like trivially copyable types, can be non-relocatable.
 - Added [table §2.1](#) directly comparing this proposal to Folly, BSL, and Qt.
 - Added straw poll results from Issaquah; removed much background and discussion (leaving only references for the interested reader to consult [\[P1144R6\]](#)).
 - Jonathan Hopkins points out that (contrary to the English description in [\[N2271\]](#)) EASTL defines `has_trivial_relocate` as a synonym for `is_trivially_move_assignable`, thus it's not prior art for this

notion. (In particular, EASTL considers `unique_ptr<int>` non-trivially relocatable.) Remove mentions of EASTL.

- Added [Appendix E](#).
- R6:
 - Added `T std::relocate(T *source)`.
 - Changed the exception guarantee of `uninitialized_relocate` and `uninitialized_relocate_n` in accord with feedback from David Stone.
 - Removed `std::ranges::relocate_at` (but kept `std::relocate_at`).
- R5 (post-Prague 2020):
 - Polymorphic types (those with any vptr at all) inhibit "natural" trivial relocatability. David Stone gave the motivating example. See [Polymorphic downcast effectively relies on offset-into-self](#).
 - Updated `concept relocatable`. Added semantic requirements on assignment and some discussion of the issue with `vector::insert`.
- R3 (post-Kona 2019):
 - User-provided copy constructors and copy/move assignment operators inhibit "natural" trivial relocatability. This permits optimizing `vector::insert` and `vector::erase`.
 - Adopted `[[trivially_relocatable(bool)]]`.
 - Added `std::relocate_at(source, dest)`.
 - Removed the core-language blanket wording that permitted "the implementation" to eliminate moves and destroys of trivially relocatable types. Instead, the existence of `std::relocate_at` and handwaviness of e.g. `[vector.capacity]` combine to strongly imply (but still not to mandate) that library vendors will use `std::relocate_at` wherever it helps with performance. We no longer propose to permit eliminating moves and destroys in e.g. `return` statements, except as already permitted under the as-if rule.

§ 2. Introduction and motivation

Given an object type `T` and memory addresses `src` and `dst`, the phrase "**relocate** a `T` from `src` to `dst`" means "move-construct `dst` from `src`, and then immediately *destroy* the object at `src`."

Any type which is both move-constructible and destructible is **relocatable**. A type is **nothrow relocatable** if its relocation operation is noexcept, and a type is **trivially relocatable** if its relocation operation is trivial (which, just like trivial move-construction and trivial copy-construction, means "tantamount to `memcpy`").

In practice, almost all relocatable types are trivially relocatable: `std::unique_ptr<int>`, `std::vector<int>`, `std::string` (on libc++), `std::list<int>` (on MSVC). Examples of non-trivially relocatable types include `std::list<int>` (on libstdc++ and libc++), `std::string` (on libstdc++ and MSVC), `std::set` (everywhere). See [Appendix C: Examples of non-trivially relocatable class types](#).

P1144 allows the library programmer to **warrant** to the compiler that a resource-management type is trivially relocatable. Explicit warrants are rarely needed because the compiler can infer trivial relocatability for Rule-of-Zero types. See [§ 4.5 Trivially relocatable type \[basic.types.general\]](#).

The most important thing about P1144 relocation is that it is backward compatible and does not break either API or ABI. My intention is simply to legalize the well-understood tricks that many industry codebases ([\[BSL\]](#), [\[Folly\]](#), [\[Deque\]](#)) are already doing in practice. P1144 is not intended to change the behavior of any existing source code (except to speed it up), nor does it require major work from standard library vendors.

§ 2.1. Optimizations enabled by trivial relocatability

The following optimizations are possible according to P1144R7's notion of trivial relocatability. Here's who does these optimizations in practice:

	vector realloc	type erasure	fixed-cap move	vector <code>erase</code>	vector <code>insert</code>	r
Arthur's libc++ std::is_trivially_relocatable_v<T>	std::vector	no	N/A	not yet	std::vector	y
[BSL] bsl::IsBitwiseMoveable<T>	bsl::vector	no	?	bsl::vector	bsl::vector	A
[Folly] folly::IsRelocatable<T>	fbvector	no	proposed for small_vector	fbvector	fbvector	N
[Qt] QTypeInfo<T>::isRelocatable	QList	QVariant	QVarLengthArray	QList	QList	q

§ 2.1.1. Contiguous reallocation

Trivial relocation can be used anywhere we do the moral equivalent of `realloc`, such as in `vector<R>::reserve`.

[Bench] (presented at C++Now 2018) shows a 3x speedup on `vector<unique_ptr<int>>::resize`. [This Reddit thread](#) demonstrates a similar 3x speedup using the online tool Quick-Bench.

§ 2.1.2. Moving in-place/SBO type-erased types like `any` and `function`

Trivial relocation can be used to de-duplicate the code generated by type-erasing wrappers like `any`, `function`, and `move_only_function`. For these types, a *move* of the wrapper object is implemented in terms of a *relocation* of the contained object. (See for example `libc++'s std::any`.) In general, the *relocate* operation must be uniquely codegenned for each different contained type `C`, leading to code bloat. But a single instantiation suffices to relocate every *trivially* relocatable `C` in the program.

§ 2.1.3. Moving fixed-capacity containers like `static_vector` and `small_vector`

The move-constructor of `fixed_capacity_vector<R,N>`, can be implemented naïvely as an element-by-element *move* (leaving the source vector's elements in their moved-from state), or efficiently as an element-by-element *relocate* (leaving the source vector empty).

For a detailed analysis of this case, see [\[FixedCapacityVector\]](#).

`boost::container::static_vector<R,N>` currently implements the naïve element-by-element-move strategy, but after LEWG feedback, [\[P0843R5\]](#) `static_vector` does [permit](#) the faster relocation strategy.

§ 2.1.4. Contiguous insert and erase

Trivial relocation can be used anywhere we shift a contiguous range left or right, such as in `vector::erase` (i.e., destroy the erased elements and "close the window" by memmoving left) and `vector::insert` (i.e., "make a window" by memmoving right and then construct the new elements in place). [\[Folly\]](#)'s `fbvector` is prior art for this optimization; see [fbvector::make_window](#).

[§ 5.2 Confusing interactions with std::pmr](#) determines whether this optimization will be possible in Standard C++.

§ 2.1.5. Swap

Given a reliable way of detecting trivial relocatability, we can optimize any routine that uses the moral equivalent of `std::swap`, such as `std::rotate`, `std::partition`, or `std::sort`.

Optimizing `std::swap` produces massive code-size improvements for all swap-based algorithms, including `std::sort` and `std::rotate`. See [@19:56–21:22](#) in my C++Now 2019 talk, and see [this Godbolt](#). [This Quick-Bench](#) shows a 25% speedup in `std::rotate` when it is allowed to use bitwise swap on a Rule-of-Zero type.

But [§ 5.2 Confusing interactions with std::pmr](#) determines whether this optimization will be possible in Standard C++.

§ 2.2. Assertions, not assumptions

Some data structures might reasonably assert the trivial relocatability of their elements, just as they sometimes assert the stronger property of trivial *copyability* today. This might help them guarantee reasonable performance, or guarantee exception-safety.

§ 2.3. Eliminate manual warrants, improve safety

Many real-world codebases contain templates which require trivial relocatability of their template parameters, but cannot today *verify* trivial relocatability. For example, [\[Folly\]](#) requires the programmer to warrant the trivial relocatability of any type stored in a `folly::fbvector`:

```
class Widget {
    std::vector<int> lst_;
};

folly::fbvector<Widget> vec; // FAILS AT COMPILE TIME for lack of warrant
```

This merely encourages the programmer to add the warrant and continue. An incorrect warrant will be discovered only at runtime, via undefined behavior. See [Allocated memory contains pointer to self](#), [\[FollyIssue889\]](#), and (most importantly) [@27:26–31:47](#) in my C++Now 2019 talk.

```
class Gadget {
    std::list<int> lst_;
};
// sigh, add the warrant on autopilot
template<> struct folly::IsRelocatable<Gadget> : std::true_type {};

folly::fbvector<Gadget> vec; // CRASHES AT RUNTIME due to fraudulent warrant
```

If this proposal is adopted, then Folly can start using `static_assert(std::is_trivially_relocatable_v<T>)` in the implementation of `fbvector`, and the programmer can stop writing explicit warrants. Finally, the programmer can start writing assertions of correctness, which aids maintainability and can even find real bugs. [Example](#):

```
class Widget {
    std::vector<int> lst_;
};
static_assert(std::is_trivially_relocatable_v<Widget>); // correctly SUCCEEDS

class Gadget {
    std::list<int> lst_;
};
static_assert(std::is_trivially_relocatable_v<Gadget>); // correctly ERRORS
```

The improvement in user experience and safety in real-world codebases ([\[Folly\]](#), [\[BSL\]](#), [\[Qt\]](#), etc.) is the most important benefit to be gained by this proposal.

§ 3. Design goals

Every C++ type already *is* or *is not* trivially relocatable. This proposal does not require any library vendor to make any library type trivially relocatable. (We assume that quality implementations will do so on their own.)

The optimizations above are in the domain of library writers. If you're writing a vector, and you detect that your element type `T` is trivially relocatable, then whether you optimize in that case is up to you. This proposal does not require any library vendor to guarantee that any particular optimization happens. (We assume that quality implementations will do so on their own.)

What C++ lacks is a standard way for library writers to *detect* the (existing) trivial relocatability of a type `T`, so that they can reliably apply their (existing) optimizations. All we really need is to add *detection*, and then all the optimizations described above will naturally emerge without any further special effort by WG21.

There are three kinds of object types that we want to make sure are correctly detected as trivially relocatable. These three cases are important for improving the performance of the standard library, and for improving the correctness of libraries such as [Folly](#)'s `fbvector`.

§ 3.1. Standard library types such as `std::string`

In order to optimize `std::vector<std::string>::resize`, we must come up with a way to achieve

```
#include <string>
static_assert(std::is_trivially_relocatable_v< std::string >);
```

This could be done unilaterally by the library vendor — via a non-standard attribute (`[[clang::trivially_relocatable]]`), or a member typedef with a reserved name, or simply a vendor-provided specialization of `std::is_trivially_relocatable<std::string>`.

That is, we can in principle solve §3.1 while confining our "magic" to the headers of the implementation itself. The programmer doesn't have to learn anything new, so far.

§ 3.2. Program-defined types that follow the Rule of Zero

In order to optimize the SBO `std::function` in any meaningful sense, we must come up with a way to achieve

```
#include <string>
auto lam2 = [x=std::string("hello")]{};
static_assert(std::is_trivially_relocatable_v< decltype(lam2) >);
```

Lambdas are not a special case in C++; they are simply class types with all their special members defaulted. Therefore, lambdas can use the same solution as

```
#include <string>
struct A {
    std::string s;
};
static_assert(std::is_trivially_relocatable_v< A >);
```

Here `struct A` follows the Rule of Zero: its move-constructor and destructor are both defaulted. If they were also *trivial*, then we'd be done. In fact they are non-trivial; and yet, because the type's bases and members are all of trivially relocatable types, the type as a whole *is* trivially relocatable.

§3.2 asks that we make the `static_assert` succeed without breaking the "Rule of Zero." We do not want to require the programmer to annotate `struct A` with a special attribute, or a special member typedef, or anything like that. We want it to Just Work. This is a harder problem than §3.1; it requires standard support in the core language. But it still does not require any new *syntax*.

§ 3.3. Program-defined types with non-defaulted special members

In order to optimize `std::vector<boost::shared_ptr<T>>::resize`, we must come up with a way to achieve

```
struct B {
    B(B&&); // non-trivial
    ~B(); // non-trivial
};
static_assert(std::is_trivially_relocatable_v< B >);
```

via some standard annotation applied to class type `B` (which in this example is standing in for `boost::shared_ptr`). We cannot possibly do it without annotation, because there exist examples of types that look just like `B` and are trivially relocatable (e.g. `libstdc++'s std::function`) and there exist types that look just like `B` and are *not* trivially relocatable (e.g. `libc++'s std::function`). So, without some kind of opt-in annotation, we cannot achieve our goal.

This paper proposes a standard syntax for §3.3, which in turn provides a *simple and portable* way for library vendors to implement §3.1.

§ 4. Proposed wording for C++2b

The wording in this section is relative to [WG21 draft N4910](#).

NOTE: There is no difficulty in changing the attribute syntax to a contextual-keyword syntax; the only downsides are aesthetic. We can defer that decision to the last minute, according to CWG's feedback on the final wording.

§ 4.1. Nothrow bidirectional iterator [[algorithms.requirements](#)]

Modify [[algorithms.requirements](#)] as follows:

- If an algorithm's template parameter is named `ForwardIterator`, `ForwardIterator1`, `ForwardIterator2`, or `NoThrowForwardIterator`, the template argument shall meet the *Cpp17ForwardIterator* requirements ([`forward.iterators`]) if it is required to be a mutable iterator, or model `forward_iterator` ([`iterator.concept.forward`]) otherwise.
- If an algorithm's template parameter is named `NoThrowForwardIterator`, the template argument is also required to have the property that no exceptions are thrown from increment, assignment, or comparison of, or indirection through, valid iterators.
- If an algorithm's template parameter is named `BidirectionalIterator`, `BidirectionalIterator1`, ~~`or`~~ `BidirectionalIterator2`, or `NoThrowBidirectionalIterator`, the template argument shall meet the *Cpp17BidirectionalIterator* requirements ([`bidirectional.iterators`]) if it is required to be a mutable iterator, or model `bidirectional_iterator` ([`iterator.concept.bidir`]) otherwise.
- If an algorithm's template parameter is named `NoThrowBidirectionalIterator`, the template argument is also required to have the property that no exceptions are thrown from increment, decrement, assignment, or comparison of, or indirection through, valid iterators.

§ 4.2. Relocation operation [[defs.relocation](#)]

Add a new section in [[intro.defs](#)]:

relocation operation

the homogeneous binary operation performed by `std::relocate_at`, consisting of a move construction immediately followed by a destruction of the source object

§ 4.3. `relocate_at` and `relocate` [[specialized.relocate](#)]

Add a new section after [[specialized.destroy](#)]:

```
template<class T>
T *relocate_at(T* source, T* dest);
```

Mandates: T shall be a complete non-array object type.

Effects: Equivalent to:

```
struct guard { T *t; ~guard() { destroy_at(t); } } g(source);
return ::new (voidify(*dest)) T(std::move(*source));
```

except that if T is trivially relocatable [*basic.types*], side effects associated with the relocation of the value of *source might not happen.

```
template<class T>
T relocate(T* source);
```

Mandates: T shall be a complete non-array object type.

Effects: Equivalent to:

```
T t = move(source);
destroy_at(source);
return t;
```

except that if T is trivially relocatable [*basic.types*], side effects associated with the relocation of the object's value might not happen.

NOTE: These functions have both been implemented in my libc++ fork; for [relocate](#), see godbolt.org/z/cqPP4oeE9 and [\[StdRelocateIsCute\]](#). My implementation of their "as-if-by-memcpy" codepaths relies on Clang's [__builtin_memcpy](#); vendors can use any vendor-specific means to implement them. The wording also deliberately permits a low-quality implementation with no such codepath at all. See [@45:23–48:39](#) in my C++Now 2019 talk.

§ 4.4. [uninitialized_relocate](#), [uninitialized_relocate_n](#), [uninitialized_relocate_backward](#) [[uninitialized.relocate](#)]

Add a new section after [\[uninitialized.move\]](#):


```
template<class InputIterator, class NoThrowForwardIterator>
NoThrowForwardIterator uninitialized_relocate(InputIterator first, InputIterator last,
                                           NoThrowForwardIterator result);
```

Effects: Equivalent to:

```
try {
    for (; first != last; ++result, (void)++first) {
        ::new (voidify(*result))
            typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*first));
        destroy_at(addressof(*first));
    }
    return result;
} catch (...) {
    destroy(++first, last);
    throw;
}
```

except that if the iterators' common value type is trivially relocatable, side effects associated with the relocation of the object's value might not happen.

Remarks: If an exception is thrown, all objects in both the source and destination ranges are destroyed.

```
template<class InputIterator, class Size, class NoThrowForwardIterator>
pair<InputIterator, NoThrowForwardIterator>
uninitialized_relocate_n(InputIterator first, Size n, NoThrowForwardIterator result);
```

Effects: Equivalent to:

```
try {
    for (; n > 0; ++result, (void)++first, --n) {
        ::new (voidify(*result))
            typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*first));
        destroy_at(addressof(*first));
    }
    return {first, result};
} catch (...) {
    destroy_n(++first, --n);
    throw;
}
```

except that if the iterators' common value type is trivially relocatable, side effects associated with the relocation of the object's value might not happen.

Remarks: If an exception is thrown, all objects in both the source and destination ranges are destroyed.

```
template<class BidirectionalIterator, class NoThrowBidirectionalIterator>
NoThrowBidirectionalIterator
uninitialized_relocate_backward(BidirectionalIterator first, BidirectionalIterator last,
                               NoThrowBidirectionalIterator result);
```

Effects: Equivalent to:

```
try {
    for (; last != first; ) {
        --last;
        --result;
        ::new (voidify(*result))
            typename iterator_traits<NoThrowBidirectionalIterator>::value_type(std::move(*last));
        destroy_at(addressof(*last));
    }
    return result;
} catch (...) {
    destroy(first, ++last);
    throw;
}
```

except that if the iterators' common value type is trivially relocatable, side effects associated with the relocation of the object's value might not happen.

Remarks: If an exception is thrown, all objects in both the source and destination ranges are destroyed.

NOTE: The Remarks allude to blanket wording in [\[specialized.algorithms.general\]/2](#).

§ 4.5. Trivially relocatable type [basic.types.general]

Add a new section in [basic.types.general]:

An object type *T* is a **trivially relocatable** type if it is:

- a trivially copyable type, or
- an array of trivially relocatable type, or
- a (possibly cv-qualified) class type declared with a `trivially_relocatable` attribute with value `true` [dcl.attr.trivrelloc], or
- a (possibly cv-qualified) class type which:
 - has no user-provided move constructors or move assignment operators,
 - has no user-provided copy constructors or copy assignment operators,
 - has no user-provided destructors,
 - has no virtual member functions,
 - has no virtual base classes,
 - all of whose members are either of reference type or of trivially relocatable type, and
 - all of whose base classes are of trivially relocatable type.

[Note: For a trivially relocatable type, the [relocation operation](#) (as performed by, for example, the library functions `std::swap` and `std::vector::resize`) is tantamount to a simple copy of the underlying bytes. —end note]

[Note: It is intended that most standard library types be trivially relocatable types. —end note]

NOTE: Polymorphic types are disallowed from "natural" trivial relocatability. See [Appendix C, example 5](#). Volatile members are not disallowed. See [\[Subobjects\]](#).

§ 4.6. [[trivially_relocatable]] attribute [dcl.attr.trivrelloc]

Add a new section after [dcl.attr.nouniqueaddr]:

The *attribute-token* `trivially_relocatable` specifies that a class type's relocation operation has no visible side-effects other than a copy of the underlying bytes, as if by the library function `std::memcpy`. It may be applied to the definition of a class. It shall appear at most once in each *attribute-list*. An *attribute-argument-clause* may be present and, if present, shall have the form

(*constant-expression*)

The *constant-expression* shall be an integral constant expression of type `bool`. If no *attribute-argument-clause* is present, it has the same effect as an *attribute-argument-clause* of (`true`).

If any definition of a class type has a `trivially_relocatable` attribute with value *V*, then each definition of the same class type shall have a `trivially_relocatable` attribute with value *V*. No diagnostic is required if definitions in different translation units have mismatched `trivially_relocatable` attributes.

If a type *T* is declared with the `trivially_relocatable` attribute, and *T* is either not move-constructible or not destructible, the program is ill-formed.

If a class type is declared with the `trivially_relocatable` attribute, and the program relies on observable side-effects of its relocation other than a copy of the underlying bytes, the behavior is undefined.

ISSUE 1 "If a type *T* is declared with the `trivially_relocatable` attribute, and *T* is either not move-constructible or not destructible, the program is ill-formed." We might want to replace this wording with a mere "Note" encouraging implementations to diagnose. See [this example](#) where a diagnostic might be unwanted.

§ 4.7. Type traits `is_relocatable` etc. [meta.unary.prop]

Add new entries to Table 47 in [meta.unary.prop]:

Template	Condition	Preconditions
template<class T> struct is_relocatable;	is_move_constructible_v<T> is true and is_destructible_v<T> is true	T shall be a complete type, cv void, or an array of unknown bound.
template<class T> struct is_nothrow_relocatable;	is_relocatable_v<T> is true and both the indicated move-constructor and the destructor are known not to throw any exceptions.	T shall be a complete type, cv void, or an array of unknown bound.
template<class T> struct is_trivially_relocatable;	T is a trivially relocatable type.	T shall be a complete type, cv void, or an array of unknown bound.

§ 4.8. `relocatable` concept [`concept.relocatable`]

Add a new section after [\[concept.copyconstructible\]](#):

```
template<class T>
concept relocatable = move_constructible<T>;
```

If T is an object type, then let rv be an rvalue of type T, lv an lvalue of type T equal to rv, and u2 a distinct object of type T equal to rv. T models `relocatable` only if

- After the definition `T u = rv;`, u is equal to u2.
- `T(rv)` is equal to u2.
- If the expression `u2 = rv` is well-formed, then the expression has the same semantics as `u2.~T(); ::new ((void*)std::addressof(u2)) T(rv);`
- If the definition `T u = lv;` is well-formed, then after the definition u is equal to u2.
- If the expression `T(lv)` is well-formed, then the expression's result is equal to u2.
- If the expression `u2 = lv` is well-formed, then the expression has the same semantics as `u2.~T(); ::new ((void*)std::addressof(u2)) T(lv);`

NOTE: We intend that a type may be `relocatable` regardless of whether it is copy-constructible; but, if it is copy-constructible then copy-and-destroy must have the same semantics as move-and-destroy. We intend that a type may be `relocatable` regardless of whether it is assignable; but, if it is assignable then assignment must have the same semantics as destroy-and-copy or destroy-and-move. The semantic requirements on assignment help us optimize `vector::insert` and `vector::erase`. `pmr::forward_list<int>` satisfies `relocatable`, but it models `relocatable` only when all relevant objects have equal allocators.

§ 5. Rationale and alternatives

§ 5.1. Attribute `[[maybe_trivially_relocatable]]`

The Clang patch currently available on Godbolt Compiler Explorer supports both `[[clang::trivially_relocatable]]` and another attribute called `[[clang::maybe_trivially_relocatable]]`, which John McCall requested that I explore.

See [P1144R4 section 6.2](#) for discussion of the `[[clang::maybe_trivially_relocatable]]` attribute, including the reasons I do not propose it for standardization.

In Issaquah (February 2023), [\[P2786R0\]](#) suggested a very similar design to `[[clang::maybe_trivially_relocatable]]`. EWGI took a three-way straw poll on the design of `[[clang::trivially_relocatable]]` versus `[[clang::maybe_trivially_relocatable]]`, with an inconclusive 7–5–6 vote (the author of P1144 voting "For" and the three representatives of P2786 presumably voting "Against."

§ 5.2. Confusing interactions with `std::pmr`

NOTE: This section was added in P1144R5 and revised in P1144R7. Here I assume libc++, where `std::string` is trivially relocatable. Feel free to substitute your favorite trivially relocatable container type; I'm sticking with `string` for these examples because it is short and easy to spell.

P1144 aims to permit efficient insertion and erasure in `std::vector`. [Example](#):

```
std::vector<std::string> vs(501);
vs.erase(vs.begin());
```

This `erase` requires us to shift down 500 `std::string` objects, which can be done either by 500 calls to `string::operator=` followed by one call to `~string`, or by one call to `~string` followed by a `memmove` (as seen for example in [Folly](#) and BSL; see [table §2.1](#)). We want to permit BSL's implementation. That's why since P1144R5, the definition of `concept relocatable` in §4.4 places semantic requirements on `T`'s assignment operators (if they exist) as well as on `T`'s constructors and destructors.

`std::pmr::polymorphic_allocator<int>` is a trivially relocatable type:

```
std::vector<std::pmr::polymorphic_allocator<int>> va;
va.emplace_back(&mr1);
va.emplace_back(&mr2);
va.emplace_back(&mr3);
va.erase(va.begin()); // A
```

Line "A" can use trivial relocation (`memmove`) because it is fast and correct: after line "A" we will have `vv[0].resource() == &mr2`. But an STL container *using* a `polymorphic_allocator` suddenly becomes problematic:

```
std::vector<std::pmr::string> vv;
vv.emplace_back("1", &mr1);
vv.emplace_back("2", &mr2);
vv.emplace_back("3", &mr3);
vv.reserve(1000); // B
vv.erase(vv.begin()); // C
```

Line "B" can certainly use trivial relocation (`memcpy`) because it is fast and correct. But on line "C", the difference between `pmr::string::operator=` and `memmove` is detectable: the former makes `vv[0].get_allocator().resource() == &mr1` and the latter would make `vv[0].get_allocator().resource() == &mr2`. Finally, hypothetically, if `vector::erase` were implemented in terms of a single `std::rotate` followed by a call to `~string`, we'd have UB here because it is UB to `swap` strings with unequal allocators.

For now, P1144R7's proposed wording implies that `std::pmr::string` must not be marked as trivially relocatable. But if we add some blanket wording setting appropriate preconditions on `std::pmr::string` (for example, permitting `vector::erase` and `deque::erase` to be implemented in terms of `rotate`, or forbidding STL containers from containing PMR types with unequal allocators), then vendors could make `std::pmr::string` trivially relocatable.

ISSUE 2 What new wording is needed to achieve this?



NOTE: Regardless, `std::pmr::polymorphic_allocator<int>` itself is trivially relocatable. The problem above arises from `propagate_on_move_assignment` etc., which affect the behavior of the *container's* `operator=`. The author of the container makes the choice whether to respect POCMA/POCCA/POCS, and also makes the choice when to warrant trivial relocatability. These choices are correlated, and so it is natural that they should be made by the same person, at the same place in the source code.

§ 5.3. Overlapping base-class subobjects

NOTE: This section was added in P1144R7.

In the following snippet, `struct Dangerous` fails to be trivially relocatable in practice because its assignment operator doesn't overwrite all `sizeof(Dangerous)` bytes of the destination object. The tail padding of `Dangerous` is allowed to contain the value of `Derived::k`. (Godbolt.)

```
struct Dangerous {
    Dangerous(int i, int j) : i(i), j(j) {}
    int i;
    short j;
};
static_assert(std::is_standard_layout_v<Dangerous>);
static_assert(std::is_trivially_relocatable_v<Dangerous>);

struct Derived : Dangerous {
    Derived(int i, int j, int k) : Dangerous(i, j), k(k) {}
    short k;
};
static_assert(!std::is_standard_layout_v<Derived>);
static_assert(!std::is_trivially_relocatable_v<Derived>);

int main() {
    Derived a = Derived(1,2,3);
    Derived b = Derived(4,5,6);
    Dangerous &ra = a, &rb = b;
    std::swap(ra, rb);
    assert(a.k == 3 && b.k == 6);
}
```

[Subspace] presented a version of this snippet, which led to the suggestion to consider only standard-layout types "naturally" trivially relocatable. Unfortunately, that criterion affects only `Derived`, whereas it's `Dangerous` that we can't trivially swap.

Possible solution #1 is due to [Subspace]: Bring the ABI notion of "data size" into standard C++, so that we can say that the "data size" of `Dangerous` is less than its "size." Types with "data size" less than "size" aren't naturally trivially relocatable. But this is painful, because it makes `is_trivially_relocatable_v<Dangerous>` false.

Possible solution #2: Don't expose the ABI notion of "data size," but do take ABI into account when deciding whether `Dangerous` is naturally trivially relocatable. This makes `is_trivially_relocatable_v<Dangerous>` implementation-defined in theory (false in practice) —even though it is trivially copyable! The difficulty here is crafting the wording.

Possible solution #3: Bring the ABI notion of "data size" into standard C++, and make the trivial version of `std::swap(T&, T&)` swap only `datasizeof(T)` bytes of memory instead of `sizeof(T)` bytes. This makes it difficult for the vendor to benefit from trivial relocation in `rotate` (because just calling `swap` won't swap all the bytes, which makes it impossible for the optimizer to coalesce adjacent memcpys into one big memcpys). This makes `is_trivially_relocatable_v<Dangerous>` true, makes `std::swap(ra, rb)` trivial, and makes `std::rotate` non-trivial.

Possible solution #4: Disallow `std::swap(T&, T&)` from using trivial relocation unless it can prove that it's operating on complete objects. This would also disallow e.g. `std::rotate` from using trivial relocation unless it's operating on complete objects (e.g. because it received a contiguous range). This creates fewer surprises, but makes it difficult for the vendor to benefit from trivial relocation in `rotate` (because they can't just get it for free by calling `swap`; they'd have to create a second entrypoint `__swap_complete_objects` and figure out how to call that instead of `swap` from algorithms that would benefit from it). This makes `is_trivially_relocatable_v<Dangerous>` true, makes `std::swap(ra, rb)` non-trivial, and makes `std::rotate` trivial with heroics.

Possible solution #5 is what [P1144R6] implicitly assumed: Put a precondition on `swap(T&, T&)` (both the template and user-defined ADL overloads) that says it's UB if either argument refers to an overlapping subobject. This makes

`is_trivially_relocatable_v<Dangerous>` true, makes `std::swap(ra, rb)` undefined behavior, and makes `std::rotate` trivial. The difficulty here is getting WG21 to accept a new precondition on `swap`.

§ 6. Acknowledgements

Thanks to Pablo Halpern for [N4158], to which this paper bears a striking resemblance—including the meaning assigned to the word "trivial," and the library-algorithm approach to avoiding the problems with "lame duck objects" discussed in the final section of [N1377]. See discussion of N4034 at Rapperswil (June 2014) and discussion of N4158 at Urbana (November 2014).

Significantly different approaches to this problem have previously appeared in Rodrigo Castro Campos's [N2754], Denis Bider's [P0023R0] (introducing a core-language "relocation" operator), and Niall Douglas's [P1029R3] (treating trivial relocatability as an aspect of move-construction in isolation, rather than an aspect of the class type as a whole). A less different approach is taken in Mungo Gill & Alisdair Meredith's [P2786R0].

Thanks to Elias Kosunen, Niall Douglas, John Bandela, and Nicolas Lesser for their feedback on early drafts of P1144R0.

Thanks to Nicolas Lesser and John McCall for their review comments on the Clang implementation [D50119].

Many thanks to Matt Godbolt for allowing me to install my Clang fork on Compiler Explorer (godbolt.org). See also [Announcing].

Thanks to Howard Hinnant for appearing with me on [CppChat], and to Jon Kalb and Phil Nash for hosting us.

Thanks to Marc Glisse for his work integrating a "trivially relocatable" trait into GNU libstdc++ and for answering my questions on GCC bug 87106.

Thanks to Jens Maurer for his feedback on P1144R3 at Kona 2019, and to Corentin Jabot for championing P1144R4 at Prague 2020.

Thanks to Dana Jansens for her contributions re overlapping and non-standard-layout types (see [Subspace]), to Alisdair Meredith for our extensive discussions during the February 2023 drafting of [P2786R0], to Giuseppe D'Angelo and Thiago Maceira for contributing the Qt entries in table §2.1, and to Giuseppe D'Angelo for extensive review comments and discussion.

§ Appendix A: Straw polls

§ Polls taken at EWGI at Issaquah on 2023-02-10

Arthur O'Dwyer presented P1144R6. Alisdair Meredith presented [P2786R0] (which proposed a `[[maybe_trivially_relocatable]]`-style facility, and expressed it as a contextual keyword instead of an attribute). EWGI took the following straw polls (as well as polls on attribute syntax and on both papers' readiness for EWG).

	SF	F	N	A	SA
The problem presented in P1144/P2786 is worth solving.	10	8	0	0	0
The problem being introduced in P1144/P2786 should be solved in a more general way instead of as proposed.	3	0	5	6	4
The annotation should "trust the user" as in P1144R6's <code>[[trivially_relocatable]]</code> ("sharp knife"), instead of diagnosing as in P1144R6's <code>[[clang::maybe_trivially_relocatable]]</code> and P2786R0's <code>trivially_relocatable</code> ("dull knife"). Three-way poll.	—	7	5	6	—

§ Polls taken at EWGI at Prague on 2020-02-13

Corentin Jabot championed P1144R4. EWGI discussed P1144R4 and Niall Douglas's [P1029R3] consecutively, then took the following straw polls (as well as a poll on the attribute syntax).

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>We believe that P1029 and P1144 are sufficiently different that they should be advanced separately.</i>	7	3	2	0	0
<i>EWGI is ok to have the spelling as an attribute with an expression argument.</i>	3	5	1	1	0
<i>EWGI thinks the author should explore P1144 as a customizable type trait.</i>	0	0	0	9	2
<i>Forward P1144 to EWG.</i>	1	3	4	1	0

For polls taken September–November 2018, see [\[P1144R6\]](#).

§ Appendix B: Sample code

See [\[P1144R6\]](#)'s Appendix B for reference implementations of `relocate`, `relocate_at`, and P1144R6's version of the `uninitialized_relocate` library algorithm, plus a conditionally trivially relocatable `std::optional<T>`.

I have implemented the entire Standard Library using the `[[clang::trivially_relocatable]]` attribute; you can find the source code [on my GitHub](#) and explore the resulting codegen [on Godbolt Compiler Explorer](#).

§ Appendix C: Examples of non-trivially relocatable class types

§ Class contains pointer to self

This fictional `short_string` illustrates a mechanism that can apply to any small-buffer-optimized class. `libc++`'s `std::function` uses this mechanism and is thus not trivially relocatable.

However, different mechanisms for small-buffer optimization exist. `libc++`'s `std::any` achieves small-buffer optimization on a 24-byte buffer, without (necessarily) sacrificing trivial relocatability.

```
struct short_string {
    char *data_ = buffer_;
    size_t size_ = 0;
    char buffer_[8] = {};

    const char *data() const { return data_; }

    short_string() = default;
    short_string(const char *s) : size_(strlen(s)) {
        if (size_ < sizeof buffer_)
            strcpy(buffer_, s);
        else
            data_ = strdup(s);
    }
    short_string(short_string&& s) {
        memcpy(this, &s, sizeof(*this));
        if (s.data_ == s.buffer_)
            data_ = buffer_;
        else
            s.data_ = nullptr;
    }
    ~short_string() {
        if (data_ != buffer_)
            free(data_);
    }
};
```

§ Allocated memory contains pointer to self

`std::list` needs somewhere to store its "past-the-end" node, commonly referred to as the "sentinel node," whose `prev` pointer points to the list's last node. If the sentinel node is allocated on the heap, then `std::list` can be trivially relocatable; but if the sentinel node is placed within the `list` object itself (as happens on `libc++` and `libstdc++`), then

relocating the `list` object requires fixing up the list's last node's `next` pointer so that it points to the new sentinel node inside the destination `list` object. This fixup of an arbitrary heap object cannot be simulated by `memcpy`.

Traditional implementations of `std::set` and `std::map` also store a "past-the-end" node inside themselves and thus also fall into this category.

```
struct node {
    node *prev_ = nullptr;
    node *next_ = nullptr;
};
struct list {
    node n_;
    iterator begin() { return iterator(n_.next_); }
    iterator end() { return iterator(&n_); }
    list(list&& l) {
        if (l.n_.next_ l.n_.next_->prev_ = &n_; // fixup
        if (l.n_.prev_ l.n_.prev_->next_ = &n_; // fixup
        n_ = l.n_;
        l.n_ = node{};
    }
    // ...
};
```

§ Class invariant depends on `this`

The `offset_ptr` provided by [\[Boost.Interprocess\]](#) is an example of this category.

```
struct offset_ptr {
    uintptr_t value_;

    uintptr_t here() const { return uintptr_t(this); }
    uintptr_t distance_to(void *p) const { return uintptr_t(p) - here(); }
    void *get() const { return (void*)(here() + value_); }

    offset_ptr() : value_(distance_to(nullptr)) {}
    offset_ptr(void *p) : value_(distance_to(p)) {}
    offset_ptr(const offset_ptr& rhs) : value_(distance_to(rhs.get())) {}
    offset_ptr& operator=(const offset_ptr& rhs) {
        value_ = distance_to(rhs.get());
        return *this;
    }
    ~offset_ptr() = default;
};
```

§ Program invariant depends on `this`

In the following snippet, `struct Widget` is relocatable, but not trivially relocatable, because the relocation operation of destroying a `Widget` at point A and constructing a new `Widget` at point B has behavior that is observably different from a simple `memcpy`.

```
std::set<void *> registry;

struct registered_object {
    registered_object() { registry.insert(this); }
    registered_object(registered_object&&) = default;
    registered_object(const registered_object&) = default;
    registered_object& operator=(registered_object&&) = default;
    registered_object& operator=(const registered_object&) = default;
    ~registered_object() { registry.erase(this); }
};

struct Widget : registered_object {};
```


§ Polymorphic downcast effectively relies on offset-into-self

Thanks to David Stone for this example. In the following snippet, `struct Base` is relocatable, but not trivially relocatable, because its copy constructor and assignment operator do not copy the entire state of the right-hand object. (Notice that `pf` is initialized with `f`, not with a copy of `o.pf`.)

```
struct Base {
    static int f(Base*) { return 21; }
    int (*pf)(Base*);
    Base(int (*pf)(Base*) = f) : pf(pf) {}
    Base(const Base& o) : pf(f) {}
    Base& operator=(const Base&) { return *this; }
};

struct Derived : Base {
    static int f(Base *self) { return ((Derived*)self)->x; }
    Derived() : Base(f) {}
    Derived(const Derived&) = default;
    Derived& operator=(const Derived& o) { x = o.x; return *this; }
    int x = 42;
};

int main() {
    Base& d = Derived();
    Base& b = Base();
    std::swap(b, d);
    printf("%d\n", b.pf(&b));
}
```

The above snippet is isomorphic to a classically polymorphic hierarchy with virtual methods. Here is the same snippet using `virtual`:

```
struct Base {
    virtual int f() { return 21; }
};

struct Derived : Base {
    int f() override { return x; }
    int x = 42;
};

int main() {
    Base& b = Base();
    Base& d = Derived();
    std::swap(b, d);
    printf("%d\n", b.f());
}
```

This is why (since P1144R5) the compiler will not consider types with virtual methods to be "naturally" trivially relocatable.

§ Appendix D: Implementation experience

A prototype Clang/libc++ implementation is at

- github.com/Quuxplusone/llvm-project/tree/trivially-relocatable
- [godbolt.org](https://ericniebler.com/2018/07/20/godbolt/), under the name "x86-64 clang (experimental P1144)"

Side-by-side case studies of `[[trivially_relocatable]]`, `[[maybe_trivially_relocatable]]`, and `[[trivially_relocatable(bool)]]` are given in [P1144R6].

As of November 2018, libstdc++ performs the `vector::resize` optimization for any type which has manually specialized `std::__is_bitwise_relocatable`. (See [this Godbolt](https://ericniebler.com/2018/07/20/godbolt/).) Manual specialization is also the approach used by [Qt], [Folly],

and [BSL]. As of 2023-02-12, the only libstdc++ library type for which `__is_bitwise_relocatable` has been specialized is `deque`; see [Deque].

Clang trunk provides a compiler builtin type trait `__is_trivially_relocatable(T)` (see [D114732]), which is largely the same as the trait proposed in this paper. (There are slight differences; e.g., Clang reports polymorphic types, reference types, and incomplete types as trivially relocatable, whereas P1144 does not. I'm not aware that any of Clang's differences were intentional.) Clang trunk has no equivalent of the `[[trivially_relocatable]]` attribute, so `__is_trivially_relocatable(T)` is true only for trivially copyable types and types marked with the `[[clang::trivial_abi]]` attribute. As of 2023-02-12, Clang trunk has no conception of a type which is non-trivial for purposes of calls and yet is trivially relocatable. But Clang's current status is compatible with P1144 (modulo the few unintentional differences in `__is_trivially_relocatable` mentioned above).

§ Appendix E: Open questions

ISSUE 3 P1144R6 `uninitialized_relocate(In, In, Fwd)`'s spec lacks the magic either-direction-overlap-handling of `memmove`. Either-direction-overlap-handling is implementable only for contiguous iterators (which can be converted to pointers); it is not implementable for arbitrary random-access iterators. Is there ever a case where someone might want to e.g. `uninitialized_relocate(rbegin(a), rend(a), begin(b))`? Do we need a separate algorithm? [Qt] provides both `q_relocate_overlap_n(T*, size_t, T*)` with magic overlap handling, and `q_uninitialized_relocate_n(T*, size_t, T*)` without. [P2786R0] proposes only `relocate(T*, T*, T*)` with magic overlap handling. [P1144R6] proposed only `uninitialized_relocate(In, In, Fwd)` without.

ISSUE 4 In P1144R7, types with `vptrs` are never naturally trivially relocatable. Arthur claims that types with `vptrs` should be used only with inheritance, which means they never do value semantics (they slice instead), which means relocating one is always a bug, just like assigning or copying one. But [P2786R0] proposes to make polymorphic types naturally trivially relocatable. Is there any use-case for relocating polymorphic types? Is P1144 being too conservative here?

ISSUE 5 In P1144R7, `std::vector<std::pmr::vector<int>>::erase(pos)` can use trivial relocation to "close up the window" in the vector; this is also done in [Qt], [Folly], and [BSL]. The difference between closing-the-window-by-move-assignment and closing-the-window-by-memmove is detectable in a conforming C++ program. Therefore [P2786R0] proposes that relocation should not be allowed here. Is there any implementation experience of P2786R0's conservative position here? ([BSL]'s `bsl::vector` follows P1144R7's liberal position already.)

ISSUE 6 In P1144R6, `std::rotate(a, a+2, a+8)` on an array `std::pmr::vector<int> a[10]` can use trivial relocation to swap the elements. The difference between swap-by-move-assignment and swap-by-trivial-relocation is not detectable in a conforming C++ program, because swapping `pmr::vectors` with unequal allocators is UB. [P2786R0] conservatively proposes that relocation should not be allowed here, anyway. Is there any usage experience in [Qt], [BSL], [Folly], or elsewhere that bears on this question? (Arthur's libcpp fork does relocation here, but doesn't really count as usage experience since he's the only one using it.)

ISSUE 7 We need a solution for potentially overlapping subobjects; see § 5.3 Overlapping base-class subobjects.

§ Index

§ Terms defined by this specification

[relocation operation](#), in § 4.2

[trivially relocatable](#), in § 4.5

§ References

§ Normative References

[N4910]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 17 March 2022. URL: <https://wg21.link/n4910>

§ Informative References

[Announcing]

Arthur O'Dwyer. *Announcing "trivially relocatable"*. July 2018. URL: <https://quuxplusone.github.io/blog/2018/07/18/announcing-trivially-relocatable/>

[Bench]

Arthur O'Dwyer. *Benchmark code from "The Best Type Traits C++ Doesn't Have"*. April 2018. URL: <https://github.com/Quuxplusone/from-scratch/blob/095b246d/cppnow2018/benchmark-relocatable.cc>

[Boost.Interprocess]

Ion Gaztañaga. *Mapping Address Independent Pointer: offset_ptr*. 2005. URL: https://www.boost.org/doc/libs/1_67_0/doc/html/interprocess/offset_ptr.html

[BSL]

Bloomberg. *bslmf::IsBitwiseMoveable: bitwise moveable trait metafunction*. 2013–2022. URL: https://github.com/bloomberg/bde/blob/962f7aa/groups/bsl/bslmf/bslmf_isbitwisemoveable.h#L8-L48

[CppChat]

Howard Hinnant; Arthur O'Dwyer. *cpp.chat episode 40: It works but it's undefined behavior*. August 2018. URL: <https://www.youtube.com/watch?v=8u5Qi4FgTP8>

[D114732]

Devin Jeanpierre. *[clang] Mark trivial_abi types as trivially relocatable*. November 2021. URL: <https://reviews.llvm.org/D114732>

[D50119]

Arthur O'Dwyer; Nicolas Lesser; John McCall. *Compiler support for P1144R0 __is_trivially_relocatable(T)*. July 2018. URL: <https://reviews.llvm.org/D50119>

[Deque]

Marc Glisse. *Improve relocation ... (__is_trivially_relocatable): Specialize for deque*. November 2018. URL: <https://github.com/gcc-mirror/gcc/commit/a9b9381580de611126c9888c1a6c12a77d9b682e>

[FixedCapacityVector]

Arthur O'Dwyer. *P1144 case study: Moving a `fixed\_capacity\_vector`*. URL: <https://quuxplusone.github.io/blog/2019/02/22/p1144-fixed-capacity-vector/>

[Folly]

Facebook. *Folly documentation on "Object Relocation"*. URL: <https://github.com/facebook/folly/blob/master/folly/docs/FBVector.md#object-relocation>

[FollyIssue889]

Arthur O'Dwyer. *Traits.h marks std::list as trivially relocatable, but in fact it is not*. URL: <https://github.com/facebook/folly/issues/889>

[N1377]

Howard Hinnant; Peter Dimov; Dave Abrahams. *N1377: A Proposal to Add Move Semantics Support to the C++ Language*. September 2002. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1377.htm>

[N2271]

Paul Pedriana. *N2271: EASTL — Electronic Arts Standard Template Library*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2271.html#type_traits_extensions

[N2754]

Rodrigo Castro Campos. *N2754: TriviallyDestructibleAfterMove and TriviallyReallocatable (rev 3)*. September 2008. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2008/n2754.html>

[N4158]

Pablo Halpern. *N4158: Destructive Move (rev 1)*. October 2014. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4158.pdf>

[P0023R0]

Denis Bider. *P0023R0: Relocator: Efficiently Moving Objects*. April 2016. URL: <http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0023r0.pdf>

[P0843R5]

Gonzalo Brito Gadeschi. *static_vector*. July 2022. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0843r5.html>

[P1029R3]

Niall Douglas. *P1029R3: move = bitcopies*. January 2020. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1029r3.pdf>

[P1144R6]

Arthur O'Dwyer. *P1144R6: Object relocation in terms of move plus destroy*. June 2022. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1144r6.html>

[P2786R0]

Mungo Gill; Alisdair Meredith. *P2786R0: Trivial relocatability options*. February 2023. URL: <https://wg21.link/p2786r0>

[Qt]

Qt Base. February 2023. URL: <https://github.com/qt/qtbase/>

[StdRelocateIsCute]

Arthur O'Dwyer. *std::relocate's implementation is cute*. May 2022. URL: <https://quuxplusone.github.io/blog/2022/05/18/std-relocate/>

[Subobjects]

Arthur O'Dwyer. *When is a trivially copyable object not trivially copyable?*. July 2018. URL: <https://quuxplusone.github.io/blog/2018/07/13/trivially-copyable-corer-cases/>

[Subspace]

Dana Jansens. *Trivially Relocatable Types in C++/Subspace*. January 2023. URL: <https://danakj.github.io/2023/01/15/trivially-relocatable.html>

§ Issues Index

ISSUE 1 "If a type `T` is declared with the `trivially_relocatable` attribute, and `T` is either not move-constructible or not destructible, the program is ill-formed." We might want to replace this wording with a mere "Note" encouraging implementations to diagnose. See [this example](#) where a diagnostic might be unwanted. ↵

ISSUE 2 What new wording is needed to achieve this? ↵

ISSUE 3 P1144R6 `uninitialized_relocate(In, In, Fwd)`'s spec lacks the magic either-direction-overlap-handling of `memmove`. Either-direction-overlap-handling is implementable only for contiguous iterators (which can be converted to pointers); it is not implementable for arbitrary random-access iterators. Is there ever a case where someone might want to e.g. `uninitialized_relocate(rbegin(a), rend(a), begin(b))`? Do we need a separate algorithm? [Qt] provides both `q_relocate_overlap_n(T*, size_t, T*)` with magic overlap handling, and `q_uninitialized_relocate_n(T*, size_t, T*)` without. [P2786R0] proposes only `relocate(T*, T*, T*)` with magic overlap handling. [P1144R6] proposed only `uninitialized_relocate(In, In, Fwd)` without. ↵

ISSUE 4 In P1144R7, types with vptrs are never naturally trivially relocatable. Arthur claims that types with vptrs should be used only with inheritance, which means they never do value semantics (they slice instead), which means relocating one is always a bug, just like assigning or copying one. But [P2786R0] proposes to make polymorphic types naturally trivially relocatable. Is there any use-case for relocating polymorphic types? Is P1144 being too conservative here? ↵

ISSUE 5 In P1144R7, `std::vector<std::pmr::vector<int>>::erase(pos)` can use trivial relocation to "close up the window" in the vector; this is also done in [Qt], [Folly], and [BSL]. The difference between closing-the-window-by-move-assignment and closing-the-window-by-memmove is detectable in a conforming C++ program. Therefore [P2786R0] proposes that relocation should not be allowed here. Is there any implementation experience of P2786R0's conservative position here? ([BSL]'s `bsl::vector` follows P1144R7's liberal position already.) ↵

ISSUE 6 In P1144R6, `std::rotate(a, a+2, a+8)` on an array `std::pmr::vector<int> a[10]` can use trivial relocation to swap the elements. The difference between swap-by-move-assignment and swap-by-trivial-relocation is not detectable in a conforming C++ program, because swapping `pmr::vectors` with unequal allocators is UB. [P2786R0] conservatively proposes that relocation should not be allowed here, anyway. Is there any usage experience in [Qt], [BSL], [Folly], or elsewhere that bears on this question? (Arthur's libc++ fork does relocation here, but doesn't really count as usage experience since he's the only one using it.) ↵

ISSUE 7 We need a solution for potentially overlapping subobjects; see § 5.3 [Overlapping base-class subobjects](#). ↵